

Final Project Report

Topic 4 — Async Research Assistant

Team:	No official name	Date:	2026-09-19
Repo:	https://github.com/iilahe/research-assistant	Tag:	v1.0-final
Members:	@iilahe @fatimekazimli @Elmin995		

Executive Summary

We built **Researcher**, a command-line research assistant that takes one natural-language question, retrieves evidence from Wikipedia, arXiv and the open web *concurrently*, and returns a single answer with numbered citations pointing back at the sources it actually used. Each source fetch has its own deadline, so a slow or dead source costs one citation rather than the whole run; the answer is synthesised once through the supplied AI module, validated structurally, and stored in PostgreSQL with a cache of retrieval results. The headline concurrency number is **1.67×**: retrieval for five questions fell from 4.83 s to 2.89 s with three sources in flight — 92 % of the 1.83× ceiling the slowest single source imposes. The headline robustness story is that *partial results are a first-class outcome, not an error*: a run that loses one source still answers, and says which one, because exit 0 with a disclosed gap is more useful to a researcher than a clean failure. The most surprising finding is that end-to-end speedup is **1.00×** — retrieval is 11 % of wall-clock time and the synthesis call that follows is the other 89 %, so a phase of concurrency work is invisible to the user until the model call shrinks too. A second surprise came from the benchmark itself: our first attempt reported 4.65× by comparing real work against fast failures, which is why it now withholds a row unless every question in both sweeps was answered. If we started over, we would build the benchmark in Phase 2 — every defect worth reporting here was found by measuring, not by reading.

Contents

Executive Summary	1
1 Project Overview	1
2 Architecture	2
3 Concurrency & Performance	4
4 Robustness & Error Handling	6
5 Storage & Caching	7
6 Testing	7
7 Deployment & Reproducibility	8
8 Limitations & Future Work	9
9 Tools & Acknowledgements	10
10 References	10
A Reproducing the benchmark	10

1. Project Overview

1.1 Problem framing and scope

A researcher asking a question of the open web faces three chores that are mechanical, slow, and easy to get wrong: deciding *where* to look, issuing those lookups without waiting for each one in turn, and keeping track of which claim came from which page. Researcher automates all three and is deliberately narrow about it. It answers *one* question per invocation, from *three* named source families, and it never presents a claim it cannot attach to a numbered source.

The scope boundary is set by what the system refuses to do. It has **no HTTP API** (ADR-006) — the deliverable is a CLI program and the container publishes no port; an API would add an authentication surface, concurrency semantics and a lifecycle this topic does not ask for. It makes **no answer-quality claim**: it verifies that an answer is *internally consistent*, never that its sources support its claims (Section 8). And it offers **no second opinion** — one provider, one model — an ensemble would triple the quota cost of a demo run, the binding constraint on a free tier (Section 3.3).

The inputs are a question, a set of source names and a results-per-source limit; the output is a rendered answer, a numbered source list, a per-source retrieval report, and a process exit status (Table 1). The last is load-bearing: the program distinguishes “answered” from “answered but incomplete” from “no usable answer” from “you invoked it wrong”, because a script wrapping this tool needs that distinction and a single non-zero code would erase it.

Table 1: Exit statuses. Zero is returned for a disclosed partial answer, because a run that loses one source has still done useful work.

Code	Name	Meaning
0	EXIT_OK	Answered. May carry warnings naming failed sources; those are printed.
1	EXIT_FAILURE	No usable answer, or a required persistence write failed.
2	EXIT_USAGE	Invalid input or configuration, detected before any I/O.

1.2 Provider choice and the AI module contract

The supplied `ai/` package is a fixed contract (TOPIC.md, lines 90–94) and is imported and never modified, so the project’s job is to *use* it well rather than to extend it. It supplies two things: source fetchers (`fetch_wikipedia`, `fetch_arxiv`, `fetch_web`) returning a list of `Source` records, and `synthesize`, which takes a question and a list of sources and returns an `AnswerWithCitations`.

We run on **Gemini** via the supplied `GeminiLLM` provider on `gemini-3.1-flash-lite`, with Tavily for web search and key-free Wikipedia and arXiv for the rest. Two facts about that choice are worth recording. First, the providers in `ai/providers/google.py` fall back to `gemini-2.0-flash`, which is retired — every 2.x model returns HTTP 404, so the application configures the model explicitly rather than relying on the supplied default. Second, Gemini’s 3.x models emit *reasoning* tokens that count against the same `max_output_tokens` budget as the answer, so the synthesis budget must leave room for the reasoning before the answer begins.

One documented divergence. The supplied `ai.sources.fetch_wikipedia` cannot answer a natural-language question. It searches with the MediaWiki `opensearch` API, which prefix-matches the *entire* query against article *titles*. Measured:

Query sent	Matching mode	Titles returned
What is photosynthesis and what are its main stages?	prefix match	0
photosynthesis	prefix match	3
photosynthesis main stages	prefix match	0

Multi-word queries match nothing, so no query-rewriting heuristic recovers this — keyword derivation was our first idea and fails too, because `photosynthesis main stages` already is the derivation. Since `ai/` is immutable, `researcher/services/wikipedia.py` replaces the *search step only*, using the MediaWiki `list=search` full-text endpoint, and keeps the supplied summary endpoint and the supplied `Source` shape. The setting `WIKIPEDIA_SEARCH=fulltext|opensearch` selects between them and defaults to `fulltext`, so the supplied path stays reachable and testable rather than being deleted. This divergence is the single place where our behaviour differs from the supplied module, and it is deliberate, documented in the module docstring, and reversible by configuration.

2. Architecture

2.1 Module map

The system is a layered modular monolith with a flat package layout: `ai/` and `researcher/` sit side by side at the repository root. ADR-001 records why — `ai/` must be importable as a top-level package because the contract fixes it there, and a flat layout means the stock `python:3.14.7-slim` image works with a plain `COPY` and a `WORKDIR`, with no `pip install .` and no packaging step. The application is 4681 lines across 25 source files; the layering rule is enforced by import direction rather than by tooling, so a lower layer never imports from a higher one.

2.2 OOP design and key abstractions

Three decisions carry most of the design.

Structural typing at the seams, inheritance nowhere. The two storage seams — `SourceCacheStore` and `SessionRepository` — are `typing.Protocol` classes, and the code depends on the protocol rather than

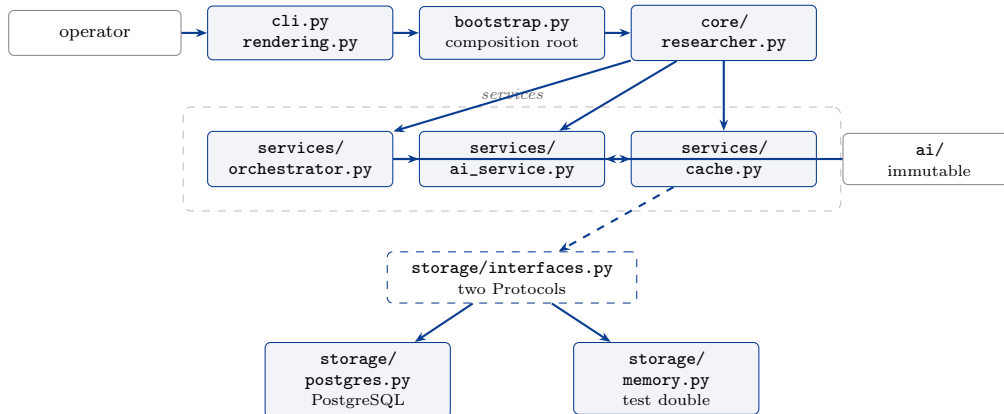


Figure 1: Module map. Solid arrows are ordinary calls; the dashed arrow into `storage/interfaces.py` is the seam where behaviour is substituted in tests. The two implementations under that seam are interchangeable because the interface is a `Protocol`, not a base class (Section 2.2).

Table 2: The modules that carry the design, and the single responsibility each owns.

Module	LOC	Responsibility
<code>models.py</code>	371	The frozen Pydantic models every layer exchanges.
<code>validation.py</code>	390	Normalising input; validating a synthesised answer.
<code>config.py</code>	316	Typed settings, derived flags, safe error messages.
<code>errors.py</code>	181	The exception taxonomy and the exit-status mapping.
<code>services/orchestrator.py</code>	249	Bounded concurrent retrieval; one outcome per source.
<code>services/ai_service.py</code>	599	The only place <code>ai/</code> is called; timeouts, retries, translation.
<code>services/cache.py</code>	199	Cache-aside reads and writes around the storage seam.
<code>services/resilience.py</code>	233	The retry policy and its backoff schedule.
<code>storage/interfaces.py</code>	146	The two <code>Protocols</code> and the error contract.
<code>storage/postgres.py</code>	293	<code>asyncpg</code> implementation; migrations applied at startup.
<code>storage/memory.py</code>	125	In-memory double; the fast test tier runs against it.
<code>core/researcher.py</code>	256	The use case: retrieve, synthesise, validate, persist.
<code>bootstrap.py</code>	284	The composition root — the only module naming concrete classes.
<code>cli.py, rendering.py</code>	584	Argument parsing, exit statuses, terminal rendering.

on a concrete implementation. Protocols beat abstract base classes for a testability reason: an ABC imposes a nominal relationship, so a test double either inherits from the real implementation or the type checker objects. A `Protocol` is satisfied structurally, which means `storage/memory.py` is a genuine second implementation that the production code cannot distinguish from `storage/postgres.py`, and `tests/test_storage.py` runs the *same* contract suite against both. That is the difference between a mock that agrees with the code and an implementation that agrees with the interface.

Frozen models at every boundary. Every value crossing a layer is a frozen Pydantic model with `extra="forbid"` — `ResearchRequest`, `Source`, `SourceOutcome`, `RetrievalResult`, `ResearchResult`, `ResearchSession`. There are no naked dictionaries in the application’s own interfaces. Frozen models give immutability for free, and `extra="forbid"` makes a typo in a keyword argument an error at construction rather than a silently ignored field — valuable in a system whose failure modes are mostly “a source quietly returned nothing”. One deliberate exception is `AnswerWithCitations`, which is the supplied module’s type and is therefore validated but not replaced.

One composition root. `bootstrap.py` is the only module that names a concrete class. Everything else receives its collaborators as constructor arguments, which is what makes the fast test tier possible: the orchestrator tests construct an orchestrator with an in-memory cache and a fake AI service, with no database, no network, and no API key. The dependency graph is assembled once, in one readable place, rather than resolved at each use site.

Two smaller abstractions earn their keep. `RetryPolicy` in `services/resilience.py` separates *whether* to retry from *what* is being retried, so the same policy covers HTTP fetches and model calls with different settings (`max_attempts=3`, exponential backoff from 0.5s capped at 8s, full jitter). And `SourceOutcome` models a per-source retrieval as a value carrying either the sources or the reason it failed, so “source X is unavailable” becomes data the renderer prints rather than an exception that unwinds the run.

2.3 Data flow: one question, end to end

A request enters at `cli.py`, which parses and validates before any I/O: an unknown source name, an empty or over-long question, or a missing credential for the selected provider is rejected here with exit 2. `bootstrap.py` then builds the object graph — settings, a shared `httpx` client, the AI service, the cache over a PostgreSQL store, and the session repository — and `ResearchService.research` asks the orchestrator for sources. The orchestrator canonicalises the query once, so two questions differing only in whitespace share a cache entry, then consults the cache per source: a hit returns the stored records with `attempts=0`, a miss acquires a slot on an `asyncio.Semaphore` and calls the AI service under `per_source_timeout_seconds`.

The AI service is the only module that touches `ai/` — it classifies the provider’s failure, decides whether the retry policy should see it, and translates it into our own `UpstreamError`. `gather(..., return_exceptions=True)` rejoins the tasks and every result, success or failure, becomes a `SourceOutcome`; a `CancelledError` is re-raised rather than converted, because swallowing it would break task cancellation. If no source produced anything the run stops with exit 1. Otherwise the AI service synthesises once under its own deadline, and `validate_answer` checks every citation index against the source list actually sent — positive, unique, in range, and not reordered after synthesis, which would silently repoint every reference number in the prose. The session is persisted as one JSON document, and the renderer prints the answer, the numbered sources and the per-source report.

3. Concurrency & Performance

3.1 Why this model

The work per question is a fan-out with no dependencies: three sources, each an independent network round trip, none needing another’s output. That is the ideal shape for `asyncio` — I/O-bound, data-independent tasks, where threads would add preemption and locking for no benefit. The whole application is single-threaded.

Two design points matter more than the choice of library. First, **the bound is a `Semaphore`, not a bigger `gather`**. Every source task acquires a slot on `asyncio.Semaphore(max_parallel_sources)`, default three. Three sources and a bound of three make that look redundant, and it is not: the bound is what keeps a request with six sources from opening six simultaneous connections to Wikipedia, and it is the knob an operator turns when a provider starts complaining — a configuration change either way, because the limit is not the size of the `gather`. Second, **there is no timeout wrapping the `gather`, deliberately**. A deadline around the group would let one slow source destroy the results of two fast ones that had already arrived; instead each task is bounded individually, so nothing that succeeded is thrown away. Total latency is therefore bounded by the per-source timeout rather than by the sum, which is the right trade for a system whose output is a citation list.

3.2 Sequential vs concurrent benchmark

`scripts/bench.py` drives the assembled application twice over the same five supplied sample questions, once with `max_parallel_sources=1` and once with the default 3, and writes `artefacts/bench.json` and a human-readable report. It measures retrieval alone and the full end-to-end path.

Table 3: Measured against live Wikipedia, arXiv, Tavily and Gemini, five sample questions, `max_results_per_source=3`. Retrieved 2026-09-16; raw output in `artefacts/bench.json`.

Workload	Sequential (bound 1)	Concurrent (bound 3)	Speedup
Retrieval, 5 questions $\times$ 3 sources	4.83 s	2.89 s	1.67$\times$
Retrieval, per question (mean)	0.97 s	0.58 s	1.67$\times$
End-to-end, 5 questions	43.4 s	43.3 s	1.00$\times$
Ceiling for three sources, $\sum t_i / \max t_i$			1.83 $\times$

The retrieval figure needs its ceiling stated, because 1.67 $\times$ looks disappointing against the naive expectation of 3 $\times$ and is not. With three independent tasks of latency t_1, t_2, t_3 , sequential execution takes $\sum t_i$ and concurrent execution takes $\max t_i$, so the best any schedule can achieve is $\sum t_i / \max t_i$ — which depends entirely on how uneven the sources are. For this source mix the ceiling is 1.83 $\times$, so the measured 1.67 $\times$ is **92 % of the achievable maximum**; the gap is scheduling overhead and the cache lookups preceding each fetch. A more even mix would show a larger ratio and a mix with one dominant source almost none — which is why the ceiling is reported beside the number.

The end-to-end row is the finding that matters, and it is the opposite of flattering. Retrieval was 4.83 s of a 43.4 s run — about 11 % — and the rest is synthesis, one sequential model call per question, untouched by the concurrency design. The honest summary of an entire phase of work is that **it optimised 11 % of the runtime**. The design is correct and the measurement is real; the number is simply small, and saying so is more useful than reporting only the row that flatters.

The first version of this benchmark lied to us. It reported a **4.65 $\times$** end-to-end speedup. The cause was not a measurement bug but a comparison bug: when Gemini’s free tier refused a synthesis, the call returned in roughly a second instead of eight, so the sequential sweep was being timed against real work while the concurrent sweep was partly timed against fast failures. The script now *withholds* the synthesis and end-to-end rows unless every question in *both* sweeps was actually answered, prints the reason, and exits 1 — so a run that cannot produce a fair comparison produces no comparison rather than a wrong one. This is the single most useful thing we built, because it found the next defect too.

3.3 Rate limits and politeness

`MAX_PARALLEL_SOURCES` bounds how many of *our* tasks are in flight. It is a politeness limit on our own behaviour and it is not a rate limit: it does not know what the provider’s quota is. What it does now, since the measurement below, is react to being throttled — the provider’s own retry instruction is honoured instead of ignored.

The free tier for the configured model permits **20 generation requests per day, per model**, and a demo run costs five syntheses — three runs exhaust the bucket. When it ran out, the provider answered with a payload naming both the limit and the remedy:

```
429 RESOURCE_EXHAUSTED ... limit: 20, model: gemini-3.8-flash
      Please retry in 21.9s
```

Listing 1: The provider’s own words, abridged. The hint is precise, and until the fix below our policy ignored all of it.

Our retry policy then waited a full-jitter backoff of at most 0.5 s between attempts, so every attempt landed inside the same throttle window: a throttle the provider described as recoverable in 22 seconds became a lost question that failed in under a second, with three attempts spent and a 30-second deadline unused.

Honouring it was not a one-line change, and the reason is a decision we would defend separately: `services/ai_service.py` *deliberately discards* the provider’s raw payload when it translates a failure, so upstream error text can never reach a log or a user. The consequence is that classification is the last place the *retry hint* exists — so that is where it is now parsed: in the exception text first, then in a `Retry-After` header, since the providers re-raise with `from` and the SDK exception’s public `.response.headers` survives on `__cause__` — no SDK import, no private attribute. An HTTP-date rather than delta-seconds is discarded, because converting one means trusting the local clock against the server’s. A new `UpstreamRateLimitError` carries the parsed delay (a number, never the payload text it came from), and the retry policy waits for the larger of its own jittered backoff and the provider’s instruction, uncapped by `max_backoff` and bounded instead by the operation’s deadline. A throttle that names no parseable delay is still classified, and still retried on the ordinary backoff; a test asserts that the measured payload’s text never appears in a message. What remains for production is a token bucket that anticipates a quota instead of discovering it through the first refused call.

4. Robustness & Error Handling

4.1 Wrapping the AI module

Everything that can fail in a third-party call is dealt with in `services/ai_service.py`, so no other layer has to know what a provider exception looks like.

Failure classification. The supplied `ProviderError` is coarse — a missing package, a missing credential and a network fault all raise the same type. The service inspects the message against a set of misconfiguration markers and separates “this can never succeed” from “this might succeed later”. The distinction is not cosmetic: retrying a fault that will never resolve spends two extra attempts, and giving up on a transient one loses a source.

Translation at the boundary. Every provider exception becomes an `UpstreamError` carrying a safe, generic message and its source. The provider’s own text is dropped on purpose — a security posture as much as a UX one, since the supplied messages can quote upstream response text verbatim. Section 3.3 shows the cost honestly: the dropped payload is exactly where the retry hint lived.

Timeouts with per-call-type retry semantics. Fetches are bounded by `per_source_timeout_seconds` and *are* retried on timeout, because a search that has not answered in ten seconds might answer in twelve. Synthesis is bounded separately by `synthesis_timeout_seconds` and is *not* retried on timeout, because the deadline wraps the whole call including its retries — a retried synthesis is guaranteed to hit the same wall after spending its budget again.

Structural validation of the result. `validate_answer` rejects non-positive, out-of-range or duplicated citation indices and detects a source list reordered between synthesis and validation. It also reports the citations the supplied `synthesize` leaves dangling: it drops out-of-range indices from the `citations` list without editing the prose, so the prose can reference a number that no longer resolves. We surface those rather than rewriting the model’s text.

4.2 Failure-mode analysis: the throttled synthesis

The incident worth documenting is the one that reached a user. During the container verification run, the demo answered three of five questions and the other two came back as “no usable answer”. The instinctive reading was a packaging fault — the image was new, the host was not — so that is what we tested first, and the test refuted it: the *same* question, asked from the host against the same database, answered in 2.92s. The image was fine. The provider’s daytime quota for that model was spent, and the two failures were the two questions that arrived after it ran out.

The trace, in the order the layers see it:

1. Gemini returns 429 `RESOURCE_EXHAUSTED` carrying the limit, the model name and a retry hint of 21.9s. The provider wrapper raises `ProviderError` with that payload folded into its message.
2. `ai_service` classifies the failure: not a misconfiguration marker, therefore retryable. `RetryPolicy` retries twice more, waiting a jittered ≤ 0.5 s each time, and both attempts land inside the same throttle window.
3. The attempts are exhausted in under a second. The error becomes an `UpstreamError` with a safe message, and the provider’s hint is discarded along with the payload.
4. The orchestrator converts it into a `SourceOutcome` carrying a failure reason. Synthesis for that question has no sources to work from, so it is reported unanswered.
5. The run continues: the other three questions answered normally, the CLI printed which failed, and the process exited 1 — the status distinguishing “some questions failed” from “the program broke”.

What went right is that the failure was contained and legible: one question was lost, not the run; the user was told which; and the exit status distinguished “some questions failed” from “the program broke”. What went wrong is entirely in steps 3–5: a condition the provider called recoverable in 22 seconds was treated as unrecoverable in one, because classification asks whether a failure is the *kind* that can be retried, never *when*.

4.3 Input validation and exit statuses

Validation happens in two places, and the split is deliberate. `validation.py` handles the *user’s* input: source names are normalised and checked against the enumeration, questions must be non-empty after canonicalisation (a question of pure punctuation is rejected), the length cap is enforced, and a credential the selected provider requires is checked for at *startup* rather than at the first call — so a misconfigured run fails in milliseconds with a clear message instead of after a timeout with a provider error. Configuration errors raise `ConfigurationError` and exit 2.

`validate_answer` handles the *model’s* output. It rejects an empty or whitespace-only answer outright: a truncated response or a safety filter would otherwise be reported as a success, because an answer with no prose

tends to have no citations either, and an empty citation list is vacuously consistent. It then checks citation structure — every index positive, in range, unique, and pointing at the source in that position. Content without citations is still accepted, as a worse answer rather than a missing one. Neither check establishes that a source supports a claim (Section 8).

5. Storage & Caching

5.1 Schema

ADR-002 puts *both* persistence concerns — the retrieval cache and the session store — in PostgreSQL, and `migrations/001_initial_schema.sql` is the whole of it. Two tables.

`source_cache` stores one row per cached lookup, keyed on the entire cache key — source, canonical query, provider, result limit and schema version — rather than a hash of those fields. That is deliberate for operability: with the components in separate columns an operator can *see* what is cached and why a lookup missed, which a digest column would hide. The write path is `INSERT ...ON CONFLICT` against exactly that key, so the upsert is enforced by the database rather than by a read-then-write. Three `CHECK` constraints mirror the model’s numeric bounds, and `expires_at >= fetched_at` is checked because a row that expires before it was fetched is a bug in the writer, not a user-visible state.

`research_sessions` stores the session as one JSON document holding the whole model, including the per-source outcomes and snippets — the supplied `AnswerWithCitations.to_dict()` is a presentation format that drops them, so it is not an archival record. Three columns are denormalised out of the payload so that listing recent sessions can order and page without parsing every document, and they are deliberately *not* constrained to the Python enumeration whose values they hold: duplicating a Python enum as a SQL `CHECK` would mean a new status could only be added by shipping a migration. The payload is validated against the model on read, so a bad value surfaces as a `StorageError` rather than as a wrong answer.

Migrations are discovered from the filesystem, recorded by filename with a checksum verified on every later start, so they are append-only: editing an already-applied file is refused rather than silently skipped, and every statement is idempotent, so a partially applied migration can be re-run safely. The schema is applied automatically at startup, which is why `migrations/` had to be in the container image (Section 7.1).

5.2 Caching policy

The cache is a cache-aside read in front of every source fetch. The policy is:

- **Keyed on the canonical query**, not the raw one — two questions differing only in case or whitespace share an entry, which makes a re-run cheap. It strips edge punctuation only, never a character *inside* a word, so `C` and `C#` are separate keys: over-normalising costs a silent wrong *hit*, under-normalising only a *miss*.
- **TTL of `cache_ttl_seconds`** (default 86 400 s, one day), stored as an absolute `expires_at` rather than an age, so expiry does not depend on any clock agreeing about when the row was written.
- **A hit reports `attempts=0`**, which distinguishes “this came from the cache” from “this succeeded on the first try” — without it a cached run and a fast network look identical in the output.
- **The cache is never authoritative.** A miss is a normal path, not a degradation, and a storage failure on the read path is logged and treated as a miss rather than failing the run — a broken cache must not take the application down with it.
- **Sessions are written, not cached.** Persistence is governed by a derived flag — the conjunction of the setting and a present DSN. When it is *off*, or the DSN is absent, the session is reported **skipped** and the run still exits 0. But when persistence is *on* and the write *fails*, that is exit 1 — the user asked for the record and did not get it.

`persistence_enabled` is *derived*, not stored: it is the conjunction of `persist_sessions` and a present DSN. Tests that inject an in-memory store must still set a DSN or the session write is silently skipped — guessing otherwise cost four failing tests. The lesson generalises: a flag that can be set inconsistently with the state it describes is a flag that will be.

6. Testing

6.1 Strategy and coverage

`pytest`, 380 tests, 96% statement coverage over `researcher/`, 6 764 lines of test code against 4 864 lines of application code. Coverage is measured over the application package only, and reported because it is real rather than a target. Every module is covered; the thinnest is `storage/session_repository.py` at 82%, where the uncovered lines are `asyncpg` error branches that need a database to fail the way the doubles cannot.

The suite has three tiers, and the tiers are the design.

The offline tier. The overwhelming majority of tests run with no network and no database, against `storage/memory.py` and fakes.

The guard tier. The suite’s offline guarantee is *enforced*, not intended. An autouse fixture in `tests/conftest.py` patches `socket.socket.connect` and refuses any address that is not loopback, private or link-local. It exists because of a real escape: the Phase 6 Wikipedia change left five tests patching a fetcher that was no longer being called, so they passed while quietly reaching the live API — green and wrong, the worst state a test can be in. Private ranges are permitted because the PostgreSQL tests need them — including under Compose, where the database is at `172.18.0.2`, which is how the guard’s first implementation (a fixed list of three loopback strings) was found too narrow. `tests/test_offline_guard.py` tests the guard itself, in both directions.

The integration tier. The PostgreSQL tests run the same contract suite as the in-memory double against a real server, and *skip* rather than fail when no DSN is configured — so a contributor without a database still gets a green suite.

6.2 Notable tests

Table 4: Four tests that earned their place by finding something.

Test	What it pins down
<code>test_synthesis_timeout</code>	Sets the fetch timeout high and the synthesis timeout to 0.05 s. If the two bounds were ever rejoined, the test would <i>hang</i> rather than pass — which is the correct failure mode for a regression that removes a deadline.
<code>test_offline_guard</code>	Eleven parametrised cases asserting that the guard permits <code>10.1.2.3</code> , <code>172.18.0.2</code> and <code>169.254.1.1</code> and refuses <code>8.8.8.8</code> , <code>2606:4700:4700::1111</code> and <code>example.com</code> . The guard is the only reason the rest of the suite’s offline claim is trustworthy.
<code>test_storage contract</code>	The same assertions run against the in-memory double and against real PostgreSQL. A divergence between them is a bug in one of the two, and this is the test that says which.
<code>test_bench</code>	Asserts that the benchmark <i>withholds</i> its speedup columns and exits 1 when a sweep was incomplete — i.e. the test that keeps the benchmark honest about the defect described in Section 4.2.

Two testing habits changed the code. First, *tests assert refusals as often as successes*: the public-error-message test plants a fake API key, forces a validation failure, and asserts the key does *not* appear in the message. Second, *a test that cannot fail is a liability*, and the response to one is to raise its cost of failure rather than delete it — which is why the synthesis-deadline regression test is written to hang.

7. Deployment & Reproducibility

7.1 The Docker image

`Dockerfile` is a two-stage build on `python:3.14.7-slim`. The builder installs runtime and test dependencies into `/install` with `-prefix`, and the runtime stage copies that prefix and the repository, creates a non-root `appuser`, hands it ownership of `/app`, and drops to it. There is no `EXPOSE` and no published port — a direct consequence of ADR-006, this project has no HTTP API — and a comment in the file calls that absence out so it reads as a decision rather than an oversight. The image runs the application as a module (`python -m researcher demo`), not as an installed console script, which is what the flat layout buys.

`compose.yaml` defines the two services the system actually needs: `postgres` (`postgres:18.6-alpine`, with a healthcheck and no published port) and `app`, which waits on that healthcheck and overrides `DATABASE_URL` to point at the service name. The `env_file` entry is optional, so the stack starts for someone who has not created a `.env` yet and simply has no model credential.

Building and running the image found three genuine defects, all of which the code review had missed and all of which are fixed:

1. `.dockerignore excluded migrations/`. The image built cleanly and would have died on its first run against a database, because `storage/postgres.py` derives the repository root from `__file__` and applies the schema at startup. A build-time success that guarantees a run-time failure is the worst kind of defect, and only *running* the thing finds it.
2. **The offline guard refused Compose’s database.** The guard permitted loopback as a fixed list of three strings; under Compose the database is at `172.18.0.2`, so seven PostgreSQL tests errored with “the test suite tried to reach the network”. The guard now tests the address with `ipaddress`, permitting loopback, private and link-local ranges and refusing public ones — the rule we had meant to write the first time.
3. `postgres:18.6-alpine exited 1`. PostgreSQL 18 moved to major-version-specific data directories and refuses data at `/var/lib/postgresql/data`; the volume must be mounted at `/var/lib/postgresql` — the published image tag was correct and the documented mount point was not.

Verification, as opposed to assertion: 16/16 smoke tests pass *inside the image* with no credentials, no network and no quota consumed, and the rest of the suite passes there too — 372 passed and 8 skipped, the skips being the PostgreSQL tests, which have no database to reach from inside the image. On the host, against a real PostgreSQL, the suite passes with **zero skips**; the demo answers end to end, writing a real session row from inside the container. The README records the exact commands and the date they were verified.

The clean-clone reproduction on a second machine — a fresh GitHub Codespace — is recorded in `artefacts/reproduction-codespaces-bfec78.txt`: at earlier commit `6f5407d`, 349 passed with seven PostgreSQL tests skipped, because the Codespace’s Docker-in-Docker bridge drops container-to-container traffic. The diagnosis is in the artefact, as is its second finding: an unreachable database cost sixty seconds per attempt — the driver’s default connect timeout — which is fixed here, where `PostgresStorage.connect` now fails fast.

7.2 Configuration

Configuration is a frozen `pydantic-settings` model read from the environment and a `.env` file. Two properties are worth naming. It uses `env_ignore_empty`, so `TAVILY_API_KEY=` means “not configured” rather than “configured with an empty string” — without it the value arrives as `''`, falsy but not `None`, and every `is None` check downstream would wrongly conclude a credential is present. And it validates credentials *presence* at startup but never logs their values; `config._describe` reports a field location and a reason but never Pydantic’s captured input, because those inputs are routinely API keys.

Table 5: Settings that change behaviour, with their bounds. `LLM_MODEL` also selects the provider’s per-model quota bucket, which on the free tier is the binding constraint on demo runs.

Setting	Default	Bound	Effect
<code>LLM_PROVIDER</code>	<code>anthropic</code>	<code>enum</code>	Provider, and which credential is required.
<code>LLM_MODEL</code>	<code>None</code>	<code>—</code>	Falls back to the provider’s default; selects the quota bucket.
<code>MAX_PARALLEL_SOURCES</code>	<code>3</code>	<code>1–16</code>	Semaphore width; the politeness bound.
<code>PER_SOURCE_TIMEOUT_SECONDS</code>	<code>10.0</code>	<code>> 0</code>	Fetch deadline, retries included. Cache waits sit outside it and are bounded by the same value as the pool’s <code>command_timeout</code> .
<code>SYNTHESIS_TIMEOUT_SECONDS</code>	<code>30.0</code>	<code>> 0</code>	Deadline for one synthesis, retries included.
<code>MAX_RESULTS_PER_SOURCE</code>	<code>3</code>	<code>1–20</code>	Upper bound on requested results per source.
<code>CACHE_TTL_SECONDS</code>	<code>86 400</code>	<code>≥ 0</code>	Cache lifetime; 0 disables reuse in practice.
<code>WIKIPEDIA_SEARCH</code>	<code>fulltext</code>	<code>enum</code>	Selects the documented search divergence.
<code>DATABASE_URL</code>	<code>None</code>	<code>DSN</code>	Enables persistence; absent means skipped .
<code>PERSIST_SESSIONS</code>	<code>true</code>	<code>bool</code>	False skips session writes; cache still uses a configured database.

The two timeout settings are separate for a reason a defect made concrete. They were originally one knob, and sharing it made two concerns invisible: raising the fetch budget to accommodate a slow search provider silently bought synthesis time as well, and lowering it to fail fast on a dead host silently began killing synthesis. Measured against the configured model, synthesis ran at 2.9s, 3.9s, 6.1s and 7.6s on a quiet run, then hit the shared ten-second ceiling — roughly one synthesis in six. Two settings, two deadlines.

8. Limitations & Future Work

These are ordered by how much they would change the system, not by how easy they are to fix.

- Answer correctness is not verified — this is the core limitation.** `validate_answer` establishes internal consistency and nothing more: that the reference numbers exist, are unique, and point where they claim to. A fluent, correctly-cited, *wrong* answer passes every check the system has. This is inherent to the approach rather than a defect more code removes — and it is precisely why partial results are surfaced rather than smoothed over. *Future work*: a grounding check between each claim and its cited source.
- The retry policy reacts to throttles but cannot anticipate them.** Measured: the provider once asked for 21.9s while the policy waited a jittered ≤ 0.5 s, so all three attempts landed inside the same window. Honouring the retry hint — from the exception text, and from the response header when the text names no delay (Section 3.3) — fixed the reaction; what remains is anticipation. *Future work*: a token bucket per provider — the current bound reacts to nothing.
- Retrieved web content is untrusted input fed to a language model.** Snippets come from arbitrary pages and are interpolated into the synthesis prompt, so a hostile page can carry text aimed at the model rather than the reader. We do structural work only — the prompt instructs the model to answer from the numbered sources, and we validate the shape of the result — and cannot validate that a source supports the claim attached to it. *Future work*: delimit retrieved text unambiguously as data, strip instruction-shaped markup, and add an entailment check.
- The concurrency win does not reach the user.** $1.67\times$ on retrieval is $1.00\times$ end to end, because synthesis is 89% of wall-clock time and is one sequential call. *Future work*: stream it, or overlap it with retrieval by synthesising a draft from whichever sources arrive first and refining it as the rest land — the only change that would make the concurrency work visible.
- Concurrency stops at the question boundary.** The demo runs its five questions sequentially; running them concurrently would need a second bound, since `MAX_PARALLEL_SOURCES` limits sources within a question and not questions, and would multiply provider load by five, which the free tier cannot absorb.
- PostgreSQL is configured with trust authentication.** `pg_hba.conf` is at `initdb`’s default, so authentication is not performed at all: any process that can reach port 5432 connects as any role, including the

superuser. The blast radius is bounded — the server listens only on loopback and the application role is not a superuser — but the exposure appears the moment the port is forwarded. *Future work*: `scram-sha-256`, a real password on the application role, and TLS for any remote connection.

7. **Secrets live in the process environment.** Adequate for a laptop and no more: anything that can read `.env` or the process's environment reads the keys, and they are long-lived. *Future work*: a secret manager, scoped credentials, rotation.
8. **Cache retention is implemented but not scheduled.** `purge_expired` exists and is tested, but nothing calls it on a schedule. *Future work*: a retention job or a database-side scheduled task.
9. **Dependencies are pinned and manually scanned.** The committed `artefacts/pip-audit.txt` found no known vulnerabilities on 17 September 2026. *Future work*: run the scan in CI and automate update pull requests.

The single thing we would do differently is ordering, not design: **build the benchmark in Phase 2, not Phase 7**. Every defect worth reporting here — the shared deadline, the comparison bug that invented a $4.65\times$ speedup, the discarded retry hint, the 89% synthesis share — was found by *measuring* the running system, and every one of them had been read past in review. The architecture held up; only the schedule was wrong.

9. Tools & Acknowledgements

Built with Python 3.14.7. Runtime dependencies are `pydantic v2` and `pydantic-settings`, `httpx`, `asynpg` and `tenacity`; development adds `pytest` with `pytest-asyncio` in strict mode and `respx` for HTTP-level fakes, `ruff` for linting and formatting, and `mypy` in strict mode over `researcher/` and `scripts/`. PostgreSQL 18.6 runs locally, and the container path was verified against `postgres:18.6-alpine`. The supplied `ai/` package is imported and never modified, so its behaviour is auditable against the original by diff.

The six Architecture Decision Records in `docs/architecture.md` carry the reasoning behind the choices this report argues for. `docs/security.md` records the security posture and the gaps, checked against the running system rather than asserted.

Table 6: AI-tool disclosure. Every entry is a task where the tool's output was verified against the code or the running system before it was accepted.

Tool	Used for
Claude (Anthropic)	Scaffolding, test-suite construction, refactoring, and drafting the documentation and this report. Verified by running the tests, the container and the benchmark, and by reading every generated module.
GitHub Copilot	Not used.
OpenAI Codex	Final audit and model-default change; checked with live providers, PostgreSQL, and rebuilt PDFs.

The AI assistants were used as a tool, not an authority: the defects here were found by executing the system, and each fix was verified by a test that fails without it. All architectural decisions, and any error remaining in this document, are the author's.

10. References

1. `ai/` — the supplied AI module contract, imported unmodified. See `TOPIC.md`, lines 90–94.
2. Project repository: <https://github.com/iilahe/research-assistant> — README, ADRs, security notes, and the benchmark artefacts.
3. `artefacts/bench.json` and `artefacts/bench-report.txt` — the raw benchmark output behind Table 3.
4. Python Software Foundation, *asyncio* — *Asynchronous I/O*, Python 3.14 documentation. <https://docs.python.org/3/library/asyncio.html>
5. Pydantic, *Models*, Pydantic v2 documentation. <https://docs.pydantic.dev/latest/concepts/models/>
6. PostgreSQL Global Development Group, *PostgreSQL 18 Documentation*. <https://www.postgresql.org/docs/18/>
7. Docker, *Multi-stage builds*. <https://docs.docker.com/build/building/multi-stage/>
8. Google, *Gemini API rate limits*. <https://ai.google.dev/gemini-api/docs/rate-limits>

A. Reproducing the benchmark

The full procedure is in `README.md`. The default benchmark makes ten measured synthesis calls and two warm-ups, plus retries; it needs live credentials and consumes quota, and it has no retrieval-only mode.

```
# 1. Dependencies and a database
python -m venv .venv && . .venv/bin/activate
pip install -r requirements.txt -r requirements-dev.txt
cp .env.example .env          # edit: LLM_PROVIDER=gemini, LLM_MODEL, keys
# 2. Full benchmark: 5 questions x 2 modes + 2 warm-ups
python scripts/bench.py --pause 10 --json artefacts/bench-reproduction.json
```